RACIOCÍNIO AUTOMÁTICO PARA PLANEAMENTO

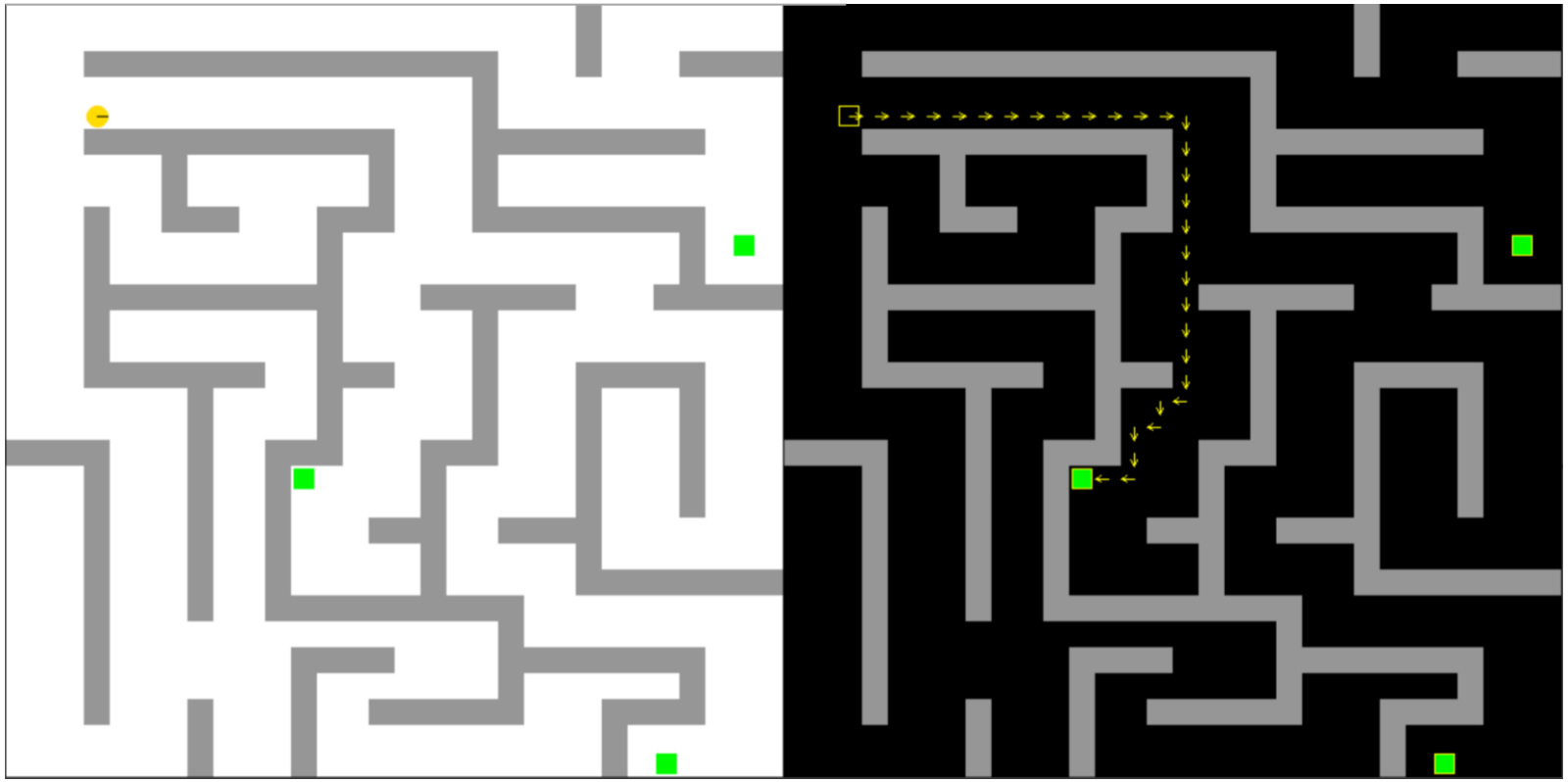
Luís Morgado

ISEL-DEI

PROBLEMAS DE PLANEAMENTO

Problemas cuja solução consiste numa sequência de acções a realizar e de situações a percorrer para, partindo de uma situação inicial, atingir uma situação final (objectivo)

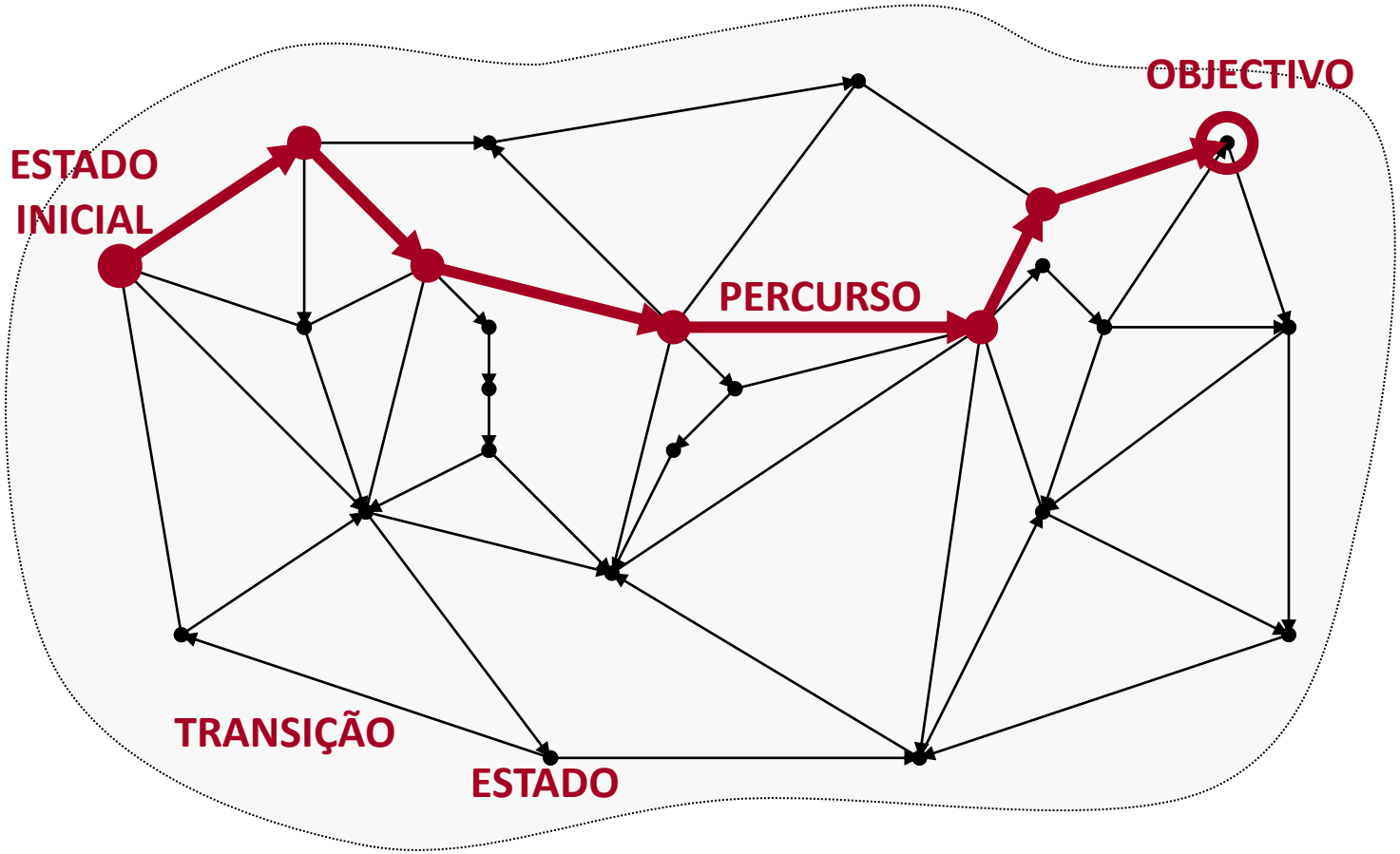
Exemplo numa plataforma de simulação:



RACIOCÍNIO ATRAVÉS DE PROCURA

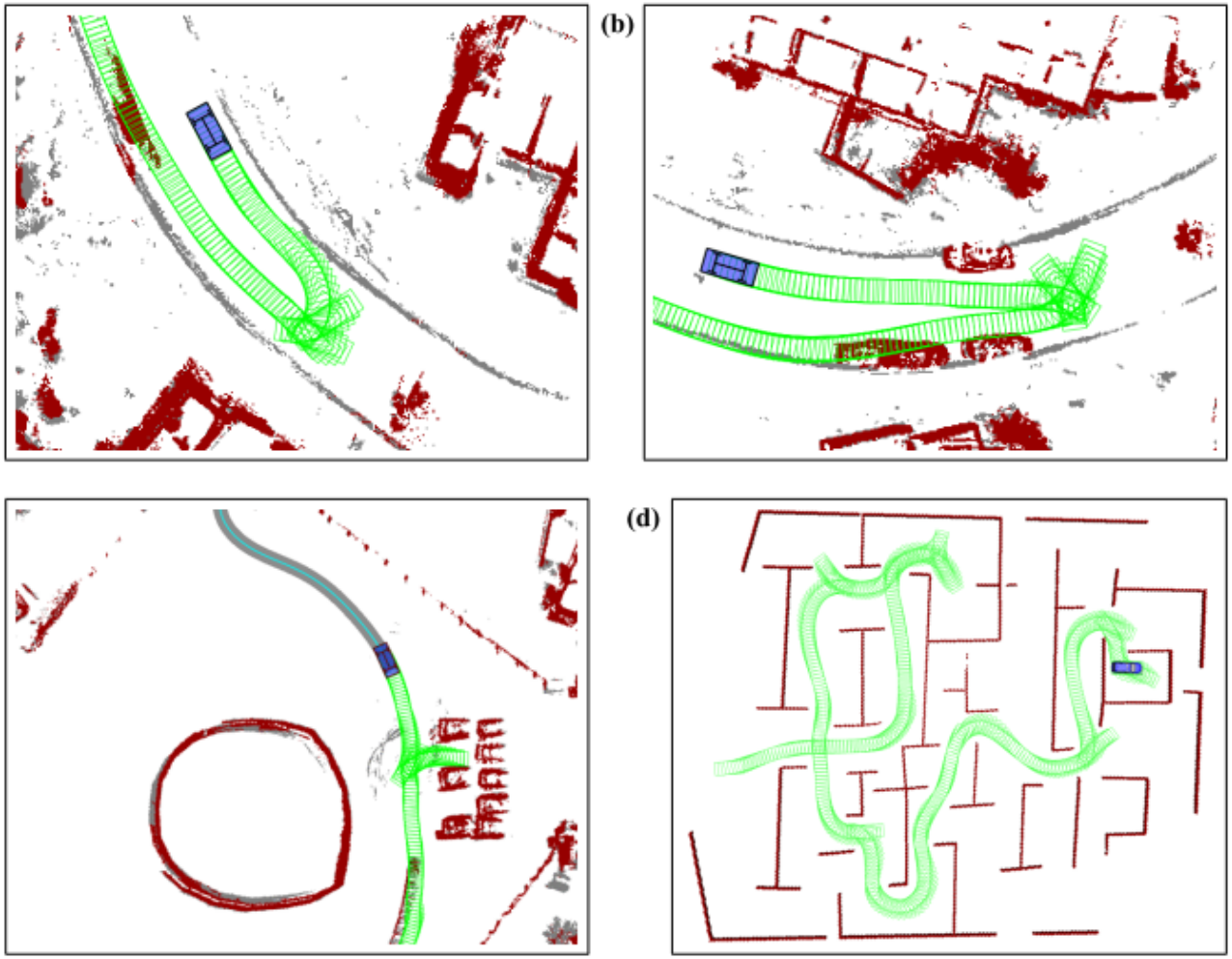
Problemas de planeamento

Objectivo: gerar um percurso do estado inicial até um estado objectivo com minimização de uma *função de custo*



PROBLEMAS DE PLANEAMENTO

Exemplo: planeamento de trajectos em veículos autónomos com base em raciocínio através de procura espaços de estados



[Montemerlo, 2008]

PLANEAMENTO FRENTE-DE-ONDA (*WAVEFRONT*)

Planeamento de trajectos em veículos autónomos

- No caso de veículos autónomos que operem em **ambientes dinâmicos** pode existir a necessidade de estratégias de acção que envolvam a **necessidade de replaneamento frequente**
- No caso de serem utilizadas **representações de ambiente discretas (em grelha)** é possível utilizar métodos de planeamento expeditos baseados na propagação de valor
- **Analogia com a propagação térmica**
  - O espaço de movimentação é modelado como um meio condutor de calor
  - O calor propaga-se a partir das posições objectivo

PLANEAMENTO FRENTE-DE-ONDA (*WAVEFRONT*)

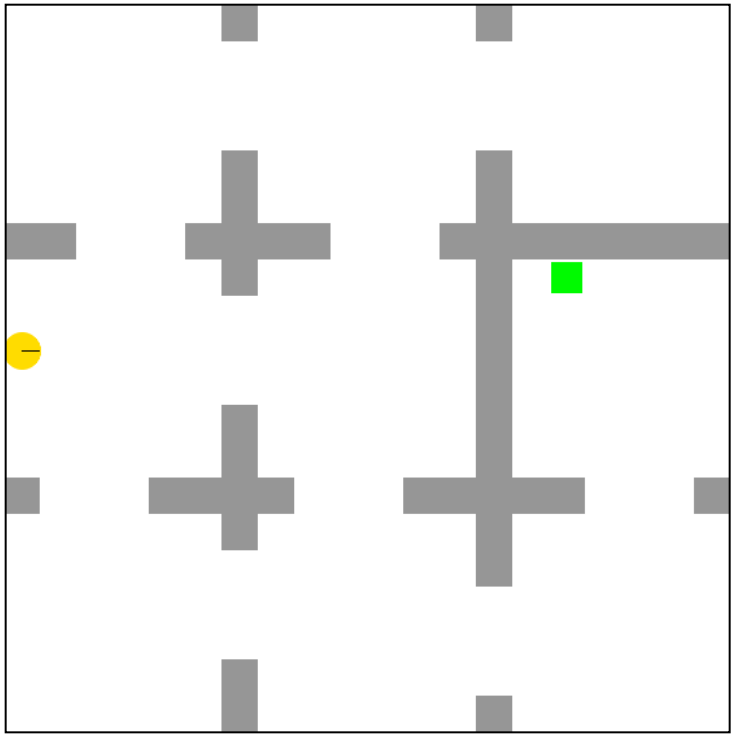
- Abordagem baseada em campos de valor
  - **Funções de valor unimodais**
    - Um único máximo (global)
  - **Campos de valor internos (motivacionais)**
    - **Simulação do mundo** com base em representações internas (**modelo do mundo**)
    - Planeamento modelado como navegação num campo de valor
      - Procura local (e.g. *hill-climbing*)
      - Estratégia global (e.g. política comportamental)

ALGORITMO FRENTE-DE-ONDA (*WAVEFRONT*)

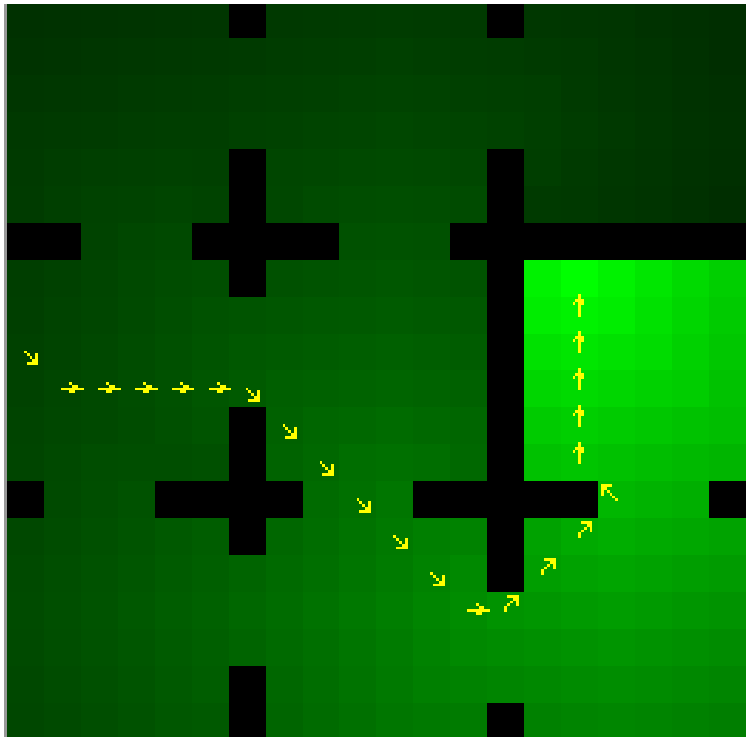
GERAR CAMPO MOTIVACIONAL INTERNO

A partir da representação (modelo) do mundo

- Estrutura
- Dinâmica (bidireccional)
- Valor - $V : S \rightarrow \mathbb{R}$



Óptimos globais



REPRESENTAÇÃO DO MUNDO

S – Estados

- Conjunto de estados do mundo

A – Acções

- Conjunto de acções possíveis

$T : S \times A \rightarrow S$ – Função de transição de estado

- Transições de estado

$Objectivos \subset S$

- Conjunto de objectivos do sistema

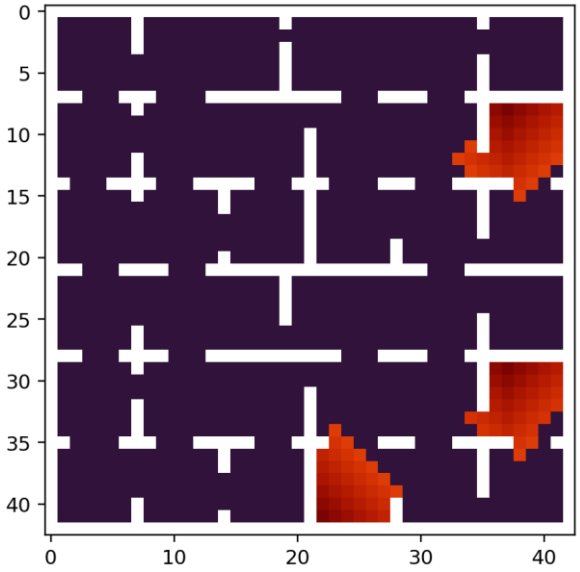
ALGORITMO FRENTE-DE-ONDA (*WAVEFRONT*)

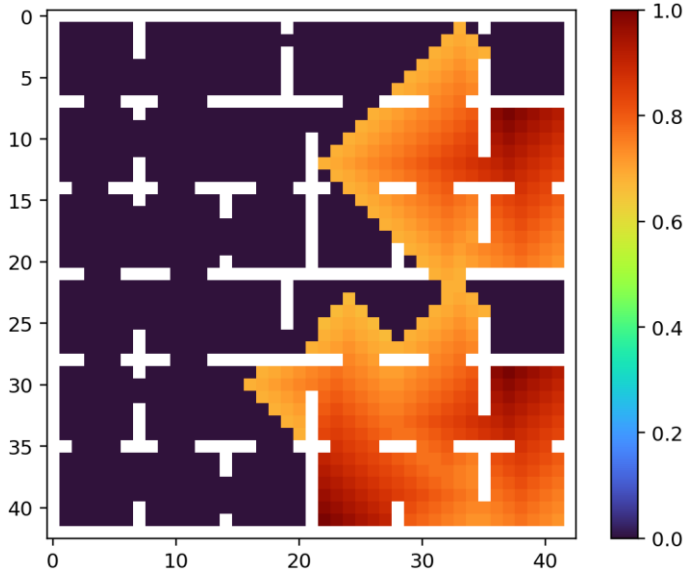
```
PROPAGAR-VALOR(objectivos)
1.  V = {}
2.  frente-de-onda = []
3.  for s in objectivos:
4.      V[s] = VALOR_MAX
5.      frente-de-onda.append(s)
6.  while frente-de-onda:
7.      s = frente-de-onda.pop(0)
8.      for s' in adjacentes(s):
9.          v = V[s] * γ dist(s, s')
10.         if v > V.get(s', -∞):
11.           V[s'] = v
12.           frente-de-onda.append(s')
```

ALGORITMO FRENTE-DE-ONDA (*WAVEFRONT*)

Exemplo: Representação do ambiente em grelha com 4 direcções de movimento

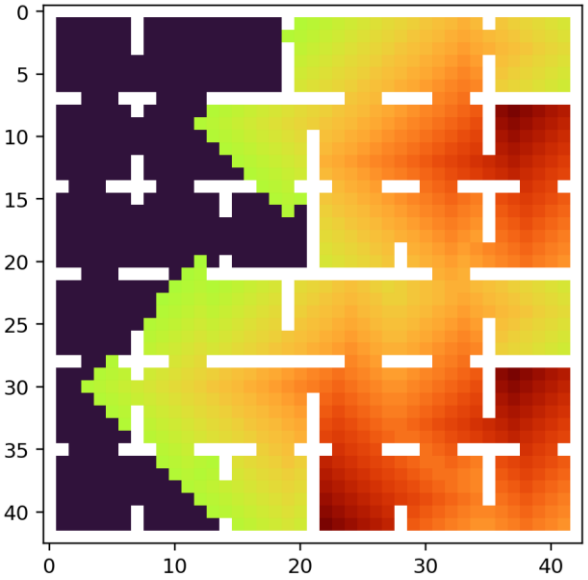
Após 100 iterações

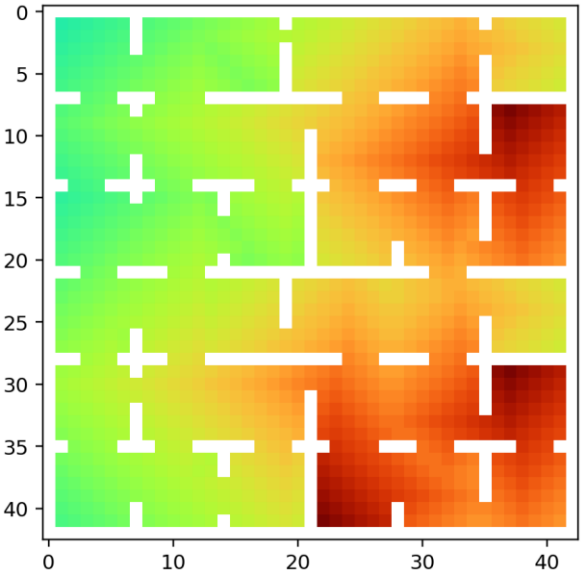




Após 500 iterações

Após 1000 iterações





Após 2000 iterações

ALGORITMO FRENTE-DE-ONDA (*WAVEFRONT*)

Planos de acção gerados com base em métodos de procura local com base no campo de valor gerado

Exemplo: Algoritmo *hill-climbing*

- Em cada passo de procura o nó actual é substituído pelo melhor nó adjacente

```
function HILL-CLIMBING(problem) returns a state that is a local maximum

  current ← MAKE-NODE(problem.INITIAL-STATE)
  loop do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then return current.STATE
    current ← neighbor
```

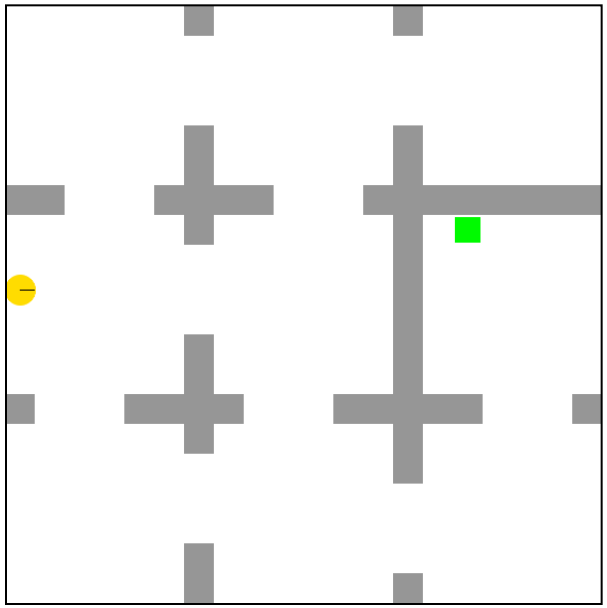
[Russel & Norvig, 2010]

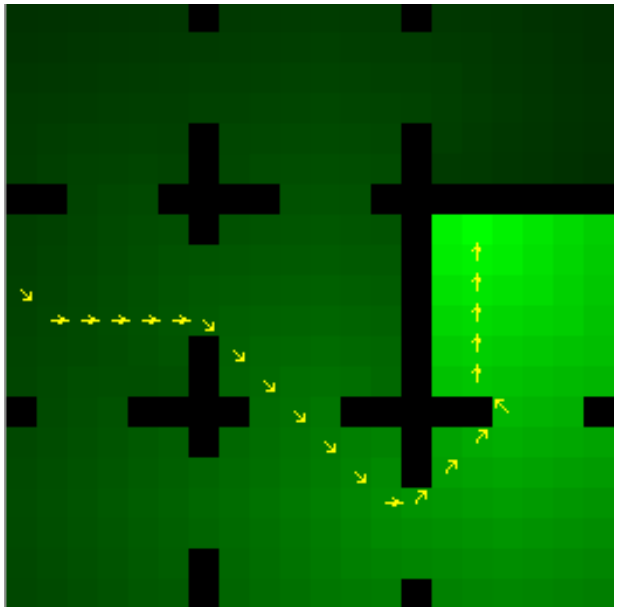
PLANO DE ACÇÃO

Planos de acção gerados com base em métodos de procura local com base no campo de valor gerado

- Não se verifica o problema da existência de óptimos locais pois a função de valor gerada é unimodal (apenas tem um máximo global)

Exemplo: Algoritmo *hill-climbing*





POLÍTICA COMPORTAMENTAL

Forma de representação do comportamento do agente para os diferentes estados possíveis

- Define qual a acção que deve ser realizada em cada estado (estratégia de acção)

S – conjunto de estados do mundo

$A(s)$ – conjunto de acções possíveis no estado $s \in S$

Política **determinista**

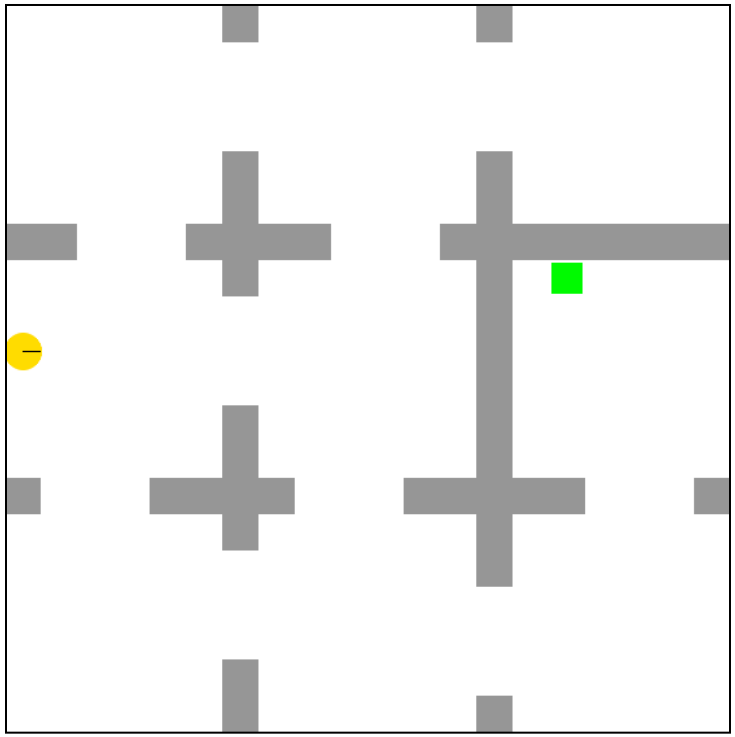
$$\pi : S \rightarrow A(s) \; ; \; s \in S$$

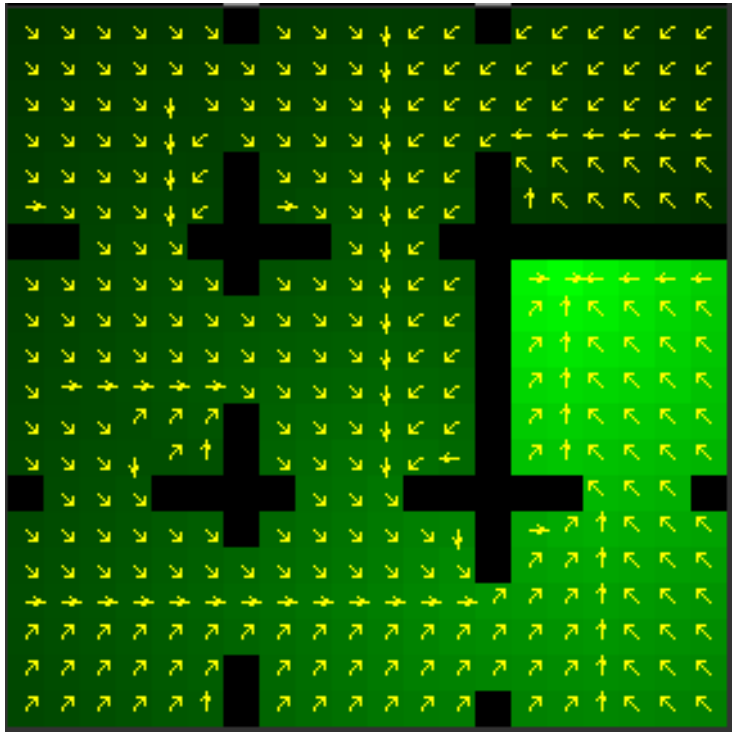
POLÍTICA COMPORTAMENTAL

Políticas de acção geradas com base no campo de valor resultante da propagação de valor

Exemplo: Política determinista

$$\pi : S \rightarrow A(s) \; ; \; s \in S$$





ALGORITMO FRENTE-DE-ONDA (*WAVEFRONT*)

Problemas:

- Utilização extensiva de memória
  - Aplicável apenas a regiões limitadas do espaço de estados
- Requer operadores bidireccionais

ALGORITMO FRENTE-DE-ONDA (*WAVEFRONT*)

Mecanismo que implementa o algoritmo *frente-de-onda*

```
1. function iniciar(γ, V_max)
2.     set γ = γ
3.     set V_max = V_max
```

ALGORITMO FRENTE-DE-ONDA (*WAVEFRONT*)

Mecanismo que implementa o algoritmo *frente-de-onda*

```
1. function propagar_valor(modelo): V
2.     V = {}
3.     frente_onda = []
4.     for s in modelo.objectivos:
5.         V[s] = V_max
6.         frente_onda.append(s)
7.     while frente_onda:
8.         s = frente_onda.pop(0)
9.         for s' in modelo.adjacentes(s):
10.            v = V[s] * γ^modelo.distancia(s, s')
11.            if v > V.get(s', -∞):
12.                V[s'] = v
13.                frente_onda.append(s')
14.    return V
```

PLANEADOR FRENTE-DE-ONDA (*WAVEFRONT*)

Iniciar planeador *frente-de-onda* definindo o valor dos parâmetros de controlo e iniciado o mecanismo *frente-de-onda*

```
1. function iniciar(modelo, γ, V_max)
2.     set modelo = modelo
3.     frente_onda = FrenteOnda(γ, V_max)
4.     V = {}
```

set propriedade = valor (atribuição de valor a propriedade)

PLANEADOR FRENTE-DE-ONDA (*WAVEFRONT*)

Planear com base num conjunto de objectivos, utilizando o mecanismo *frente-de-onda* para gerar a função valor $V(s)$ e a respectiva política

```
1. function planear(): politica
2.     V = frente_onda.propagar_valor(modelo)
3.     politica = {s: argmax(a ∈ modelo.A, valor_accao(s, a))
4.                  for s in modelo.S
5.                  if s not in objectivos}
6.    return politica
```

Estados finais (absorventes)

`valor_accao(s, a)` – Valor de no estado **s** realizar a acção ***a***

PLANEADOR FRENTE-DE-ONDA (*WAVEFRONT*)

Obter valor de num estado realizar uma acção com base na função valor $V(s)$ gerada

```
1. function valor_accao(s, a)
2.     s' = modelo.T(s, a)
3.     return V.get(s', -∞)
```

`get(chave, valor_omissão): valor` (acesso a dicionário)

BIBLIOGRAFIA

[Russel & Norvig, 1995]

S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 1st Edition, Prentice Hall, 1995

[Russel & Norvig, 2022]

S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2022

[Nilsson, 1998]

N. Nilsson , *Artificial Intelligence: A New Synthesis*, Morgan Kaufmann 1998

[Luger, 2009]

G. Luger , *Artificial Intelligence: Structures and Strategies for Complex Problem Solving* , Addison-Wesley, 2009

[Jaeger & Hamprecht, 2010]

M. Jaeger, F. Hamprecht, *Automatic Process Control for Laser Welding*, Heidelberg Collaboratory for Image Processing (HCI) , 2000

[Pearl, 1984]

J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley, 1984

[Montemerlo, 2008]

M. Montemerlo *et al.*, *Junior: The Stanford Entry in the Urban Challenge*, Stanford Artificial Intelligence Lab, 2008